

10

DISK SCHEDULING

10.1 Goal of Disk Scheduling

- Enhanced throughput.
- Minimize the average seek time of the disk.
- Fair chance to all the I/O requests.

10.2 Disk scheduling Algorithms

10.2.1 FCFS

It serves the first request in the queue.

10.2.2 SSTF

It selects the request with minimum seek time for the current head position.

10.2.3 SCAN or Elevator Algorithm

The disk arm starts from the current head position and moves towards the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and servicing continues.

10.2.4 C-SCAN

The disk arm starts from the current head position and moves towards the other end, servicing requests until it gets to the other end of the disk, where the head movement is reversed and immediately returns to the beginning of the disk without servicing any requests at the return trip. The servicing then continues from the beginning of the disk again.

10.2.5 C-LOOK

It is similar to C-SCAN where the arm goes as far as the last request in each direction, then reverses the direction immediately, without first going all the way to the end of the disk.

Note:

- Performance depends on the number and types of requests.
- I/O requests can be influenced by the file allocation methods.
- Either SSTF or LOOK is a reasonable choice for the default algorithm.

10.2.6 Comparison Among the Disk Scheduling Algorithms

Algorithm	Advantages	Disadvantages
First Come First Served	• Easy to implement • Fair chance to all the I/O requests • It doesn't suffer from starvation	• Seek time is not optimized. • It doesn't maximize throughput.
Shortest Seek Time First	• It tends to minimize the arm movement. • It has better throughput than FCFS.	• It may suffer from starvation.
SCAN/LOOK	• It eliminates starvation. • It works well with light to heavy loads.	• It is complex to implement. • Increased overhead. • It needs a directional bit to keep track of the head direction.
C-SCAN/C-LOOK	• No directional bit is required. • It works well with light to heavy loads.	• It is complex to implement. • Increased overhead.

□□□